Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase IV · Escala, sistemas y arquitecturas modernas

Módulo 18

Arquitecturas modernas

Normalización, atención eficiente, contexto largo, mezcla de expertos, modelos de espacio de estados y multimodalidad

---

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 18: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
d, L, d_{ff}	dimensión del modelo, número de bloques y dimensión intermedia del MLP
H, $d_h = d/H$	cabezas de consulta y dimensión por cabeza
n_{kv}, G	cabezas de clave y valor; número de grupos de GQA ($n_{\text{kv}} = G$)
T, b, s	longitud de la secuencia, tamaño del lote y bytes por número (2 en bf16)
d_c, d_h^R	dimensión latente de MLA y dimensión de la clave con RoPE desacoplada
W	tamaño de la ventana de atención local
θ_i, $\lambda_i = 2\pi/\theta_i$, β_0	frecuencias y longitudes de onda de RoPE; base (10^4 en el artículo original)
$s_{\text{e}} = L'/L_0$	factor de extensión del contexto, de L_0 a L' tokens
$E_1, \ldots, E_N$, k	expertos y número de expertos activos por token
$z = xW_r$, $p_i(x)$, $g_i(x)$	logits del enrutador, probabilidades y pesos de combinación
f_i, P_i, $\mathcal{L}_{\text{bal}}$	fracción de tráfico, probabilidad media y pérdida de balanceo del experto i
N_{tot}, N_{act}	parámetros totales y activos por token
A, B, C, Δ	matrices de un modelo de espacio de estados continuo y paso de discretización
$\bar{A}, \bar{B}$, h_t	matrices discretizadas y estado en el instante t
σ, $\text{softplus}(z) = \ln(1 + e^z)$	sigmoide logística y función *softplus*
P, C_{in}	lado del parche de un ViT y número de canales de la imagen

Como en el módulo 14, los vectores de tokens son filas y una capa lineal actúa como $x \mapsto xW$. Los números grandes se escriben con espacio fino como separador de miles (46 702 792 704). Las unidades binarias son 1 KiB $= 2^{10}$, 1 MiB $= 2^{20}$ y 1 GiB $= 2^{30}$ bytes; 1 GB son 10^9 bytes.

Índice

Sobre este resumen **ii**

1 Componentes de consenso del Transformer moderno **1**
1.1 El bloque de consenso . . . 1
1.2 RMSNorm . . . 2
1.3 El presupuesto del MLP con puerta . . . 3
1.4 Hiperparámetros de forma . . . 3

2 Atención eficiente en memoria: caché KV, MQA, GQA y MLA **5**
2.1 La caché de claves y valores . . . 5
2.2 La decodificación está limitada por la memoria . . . 5
2.3 Atención multiconsulta y de consultas agrupadas . . . 6
2.4 Atención latente multicabeza . . . 6

3 Atención local y contexto largo **8**
3.1 Atención con ventana deslizante . . . 8
3.2 Campo receptivo . . . 8
3.3 La caché circular . . . 9
3.4 Patrones dispersos e híbridos . . . 9
3.5 RoPE y longitudes de onda . . . 9
3.6 Extensión del contexto . . . 10

4 Mezcla de expertos **11**
4.1 La capa de mezcla de expertos . . . 11
4.2 Parámetros totales y activos . . . 11
4.3 Balanceo de carga . . . 12
4.4 Capacidad y descarte de tokens . . . 13
4.5 Expertos de grano fino y compartidos . . . 13
4.6 Casos públicos . . . 13

5 Modelos de espacio de estados **15**
5.1 Sistemas lineales continuos y su discretización . . . 15
5.2 Recurrencia y convolución . . . 15
5.3 Selección: el modelo de Mamba . . . 16
5.4 El barrido paralelo . . . 16

5.5 Comparación con la atención 17

6 Multimodalidad: imágenes y audio como tokens 18

6.1 El Vision Transformer 18

6.2 Audio 19

6.3 Fusión con el modelo de lenguaje 19

Bibliografía 20

Tema 1

Componentes de consenso del Transformer moderno

Entre el Transformer original de 2017 y los modelos abiertos actuales hay decenas de variantes publicadas, pero la mayoría converge en un mismo bloque: normalización antes de cada subcapa, RMSNorm, MLP con puerta, codificación rotatoria de la posición y ausencia de sesgos. Este tema define ese bloque de consenso, demuestra las propiedades de RMSNorm que lo justifican y estudia los hiperparámetros de forma que comparten casi todos los modelos. Se sigue la clase 3 del curso CS336 [16], el artículo de Zhang y Sennrich [33, §3–4], el de Shazeer [25, §2–3] y la tabla de hiperparámetros de Llama 3 [13, §3.2]. El bloque decoder-only, SwiGLU y RoPE ya se han estudiado en el módulo 14; aquí se recuerdan solo en la medida en que se necesitan.

1.1 El bloque de consenso

La clase 3 de CS336 compara las arquitecturas densas publicadas entre 2019 y 2025 y observa que hay poco consenso en los detalles, con una excepción casi universal: la normalización se coloca a la entrada de cada subcapa, fuera del camino residual [16]. Las demás elecciones convergen hacia lo que el curso llama arquitecturas «de tipo LLaMA».

Definición 1.1 (Bloque de consenso). Sean $h \in \mathbb{R}^{1\times d}$ la fila de un token en el flujo residual, Attn una atención multicabeza causal con RoPE y FFN un MLP SwiGLU sin sesgos. El *bloque de consenso* es

$$a = h + \mathrm{Attn}\big(\mathrm{RMSNorm}_1(h)\big), \qquad h' = a + \mathrm{FFN}\big(\mathrm{RMSNorm}_2(a)\big),$$

donde las capas lineales no tienen sesgo y la posición se introduce solo mediante la rotación de consultas y claves en cada capa. El modelo completo añade un RMSNorm final antes de la matriz de salida.

Cuadro 1.1: Elecciones de arquitectura del Transformer original y de Llama 3. Fuentes: [29, §3], [13, §3.2 y tabla 3] y [16].

Elección	Transformer (2017)	Llama 3 (2024)
Posición de la normalización	tras la subcapa (post-norm)	antes de la subcapa (pre-norm)
Normalización	LayerNorm	RMSNorm
Activación del MLP	ReLU, $d_{\mathrm{ff}} = 4d$	SwiGLU, $d_{\mathrm{ff}} \approx 3{,}5d$
Posición	sinusoidal sumada a la entrada	RoPE en cada capa, base $5 \cdot 10^5$
Sesgos	sí	no
Atención	multicabeza (MHA)	agrupada (GQA, 8 cabezas K/V)

Observación 1.2. Las justificaciones de cada elección son mayoritariamente empíricas. Shazeer termina su artículo sobre las unidades con puerta sin ofrecer explicación de por qué funcionan mejor [25, §4], y la clase 3 de CS336 insiste en que las diferencias entre variantes son pequeñas y en que el argumento a favor de RMSNorm es sobre todo de coste [16]. Lo que sí admite demostración son las propiedades algebraicas que diferencian unas variantes de otras, y a ellas se dedica la sección siguiente.

1.2 RMSNorm

Definición 1.3 (LayerNorm y RMSNorm). Sean $x \in \mathbb{R}^d$, $\gamma \in \mathbb{R}^d$ un vector de ganancias y $\varepsilon \geq 0$. Con $\mu(x) = \frac{1}{d}\sum_i x_i$ y $\mathrm{RMS}_\varepsilon(x) = \sqrt{\frac{1}{d}\sum_i x_i^2 + \varepsilon}$, se definen

$$\mathrm{RMSNorm}(x) = \frac{x}{\mathrm{RMS}_\varepsilon(x)} \odot \gamma, \qquad \mathrm{LN}(x) = \frac{x - \mu(x)\mathbf{1}}{\sqrt{\frac{1}{d}\sum_i (x_i - \mu(x))^2 + \varepsilon}} \odot \gamma + \beta,$$

donde $\odot$ es el producto componente a componente y $\beta \in \mathbb{R}^d$ es el sesgo de LayerNorm. RMSNorm no resta la media ni tiene sesgo [33, §4, ec. 4].

La práctica 1 de CS336 usa exactamente esta forma, con $\varepsilon = 10^{-5}$ y cálculo en fp32 aunque la entrada esté en bf16, para evitar desbordamientos al elevar al cuadrado [17, §3.4.1].

Proposición 1.4 (Propiedades de RMSNorm). *Sea $\varepsilon = 0$ y sea $x \neq 0$.*

(i) Norma fija. *Con $\gamma = \mathbf{1}$,* $\|\mathrm{RMSNorm}(x)\| = \sqrt{d}$.

(ii) Invariancia de escala. *Para todo $\alpha > 0$,* $\mathrm{RMSNorm}(\alpha x) = \mathrm{RMSNorm}(x)$*; para $\alpha < 0$,* $\mathrm{RMSNorm}(\alpha x) = -\mathrm{RMSNorm}(x)$.

(iii) Invariancia ante la escala de los pesos. *Si $x = uW$ es la salida de una capa lineal y W se sustituye por αW con $\alpha > 0$, la salida normalizada no cambia.*

(iv) Relación con LayerNorm. *Con $\beta = 0$,* $\mathrm{LN}(x) = \mathrm{RMSNorm}\big(x - \mu(x)\mathbf{1}\big)$ *siempre que x no sea constante.*

La proposición 1.4(iv) dice que LayerNorm es RMSNorm precedida de un centrado. Lo que RMSNorm pierde es, por tanto, la invariancia ante traslaciones: $\mathrm{LN}(x + t\mathbf{1}) = \mathrm{LN}(x)$ para todo $t \in \mathbb{R}$, mientras que $\mathrm{RMSNorm}(x + t\mathbf{1}) \neq \mathrm{RMSNorm}(x)$ en general. Zhang y Sennrich conjeturan que la invariancia ante la escala es la propiedad que importa para la estabilidad, y no la de centrado, y la tabla 1 de su artículo compara ambas normalizaciones en estos términos [33, §4.1].

Observación 1.6 (Coste). Las normalizaciones suponen una fracción despreciable de los FLOPs de un bloque, pero no de su tiempo: son operaciones limitadas por la memoria, con intensidad aritmética baja (módulo 17, tema 2). Por eso la clase 3 de CS336 atribuye a RMSNorm una ganancia de tiempo real apreciable a pesar de ahorrar muy pocas operaciones [16], y Zhang y Sennrich informan de reducciones del tiempo de ejecución de entre el 7 y el 64 % según el modelo [33].

1.3 El presupuesto del MLP con puerta

Definición 1.7 (MLP SwiGLU). Sean $W_1, W_3 \in \mathbb{R}^{d \times d_{\text{ff}}}$ y $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d}$. El MLP SwiGLU es

$$\text{FFN}_{\text{SwiGLU}}(x) = \big(\text{SiLU}(xW_1) \odot xW_3\big)W_2, \qquad \text{SiLU}(t) = t\,\sigma(t),$$

sin sesgos [25, ec. 6].

Proposición 1.8 (Presupuesto de parámetros). *Un MLP SwiGLU con dimensión intermedia d_{ff} tiene $3d\,d_{\text{ff}}$ parámetros y cuesta $6d\,d_{\text{ff}}$ FLOPs por token en la pasada hacia delante. Tiene los mismos parámetros y FLOPs que un MLP de dos matrices con dimensión intermedia $4d$ si y solo si $d_{\text{ff}} = \frac{8}{3}d$.*

Shazeer aplica esta regla en sus experimentos: compara un MLP con $d_{\text{ff}} = 3072$ y dos matrices con variantes con puerta de $d_{\text{ff}} = 2048 = \frac{2}{3} \cdot 3072$ [25, §3.1]. La clase 3 de CS336 recoge que la mayoría de los modelos con puerta usan cocientes d_{ff}/d entre 2,5 y 3,5, y que fuera de ese intervalo hay excepciones como T5, con un cociente de 64 [16]. La cuadro 1.2 muestra los valores de Llama 3, Mixtral y Grok-1.

Cuadro 1.2: Hiperparámetros de forma de modelos con publicación de su configuración. Fuentes: [13, tabla 3], [18, tabla 1] y el archivo `run.py` de [30].

Modelo	L	d	d_{ff}	H	n_{kv}	d_{ff}/d	d/L
Llama 3 8B	32	4096	14 336	32	8	3,5	128
Llama 3 70B	80	8192	28 672	64	8	3,5	102,4
Llama 3 405B	126	16 384	53 248	128	8	3,25	130,0
Mixtral 8x7B	32	4096	14 336	32	8	3,5	128
Grok-1	64	6144	32 768	48	8	5,333	96

1.4 Hiperparámetros de forma

La clase 3 de CS336 identifica tres regularidades más, todas ellas con excepciones documentadas [16].

Definición 1.9 (Hiperparámetros de forma). En un Transformer con L bloques, dimensión d y H cabezas de dimensión d_h:

(i) la *regla de las cabezas* es $Hd_h = d$;

(ii) la *relación de aspecto* es $r = d/L$;

(iii) el *tamaño del vocabulario* es $|\mathcal{V}|$.

Los modelos de la cuadro 1.2 cumplen la regla de las cabezas con $d_h = 128$, salvo Grok-1, en el que $48 \cdot 128 = 6144 = d$ también la cumple. La clase de CS336 sitúa la relación de aspecto de la mayoría de los modelos entre 100 y 200, y los vocabularios entre $3 \cdot 10^4$ y $5 \cdot 10^4$ tokens para modelos monolingües y entre 10^5 y $2{,}5 \cdot 10^5$ para los multilingües: Llama 2 usa 32 000 y Llama 3, unos 128 000 [16, 13].

Proposición 1.10 (Dimensión a partir del tamaño y la relación de aspecto). *Sea un Transformer de consenso con $d_{\text{ff}} = \frac{8}{3}d$ y atención multicabeza, cuyos bloques contienen N_{b}*

parámetros en total, y sea $r = d/L$ *su relación de aspecto. Despreciando las normalizaciones,* $N_{\mathrm{b}} = 12Ld^2$ *y, por tanto,*

$$d = \Big(\frac{rN_{\mathrm{b}}}{12}\Big)^{1/3}, \qquad L = \frac{d}{r}.$$

Observación 1.12 (Estabilidad). La clase 3 de CS336 recoge tres recursos de estabilidad frecuentes en modelos recientes, todos dirigidos a las funciones softmax: la pérdida auxiliar z sobre el logaritmo del denominador de la softmax de salida, la normalización de consultas y claves antes del producto escalar (*QK-norm*) y el recorte suave de los logits mediante una tangente hiperbólica [16]. Las tres limitan el crecimiento de los logits, cuyo exceso satura la softmax y desestabiliza el entrenamiento.

Tema 2

Atención eficiente en memoria: caché KV, MQA, GQA y MLA

Al generar texto, un Transformer produce un token cada vez y reutiliza las claves y los valores de las posiciones anteriores. Esa reutilización ahorra cómputo, pero obliga a guardar y a leer en cada paso una caché que crece con la longitud del contexto y con el tamaño del lote, y que en contextos largos ocupa más memoria que los propios pesos. Este tema cuantifica esa caché, demuestra que con atención multicabeza la decodificación está necesariamente limitada por la memoria y estudia las tres variantes que la reducen: la atención multiconsulta (MQA), la atención de consultas agrupadas (GQA) y la atención latente multicabeza (MLA). Se sigue a Shazeer [24, §2.4 y §3], a Ainslie *et al.* [1, §2], a DeepSeek-AI [6, §2.1] y la clase 10 de CS336 [16].

2.1 La caché de claves y valores

Sea una capa de atención con H cabezas de dimensión d_h. En la generación autorregresiva, al procesar el token t se necesitan las consultas de t y las claves y los valores de las posiciones $1, \dots, t$. Por la causalidad (módulo 14, tema 1), las claves y los valores de las posiciones $j < t$ no dependen de t: se calculan una vez y se guardan.

Definición 2.1 (Caché KV). La *caché KV* de una capa, tras procesar T tokens, es el par de tensores (K, V) con las claves y los valores de las T posiciones para cada una de las n_{kv} cabezas de clave y valor. Cada tensor tiene $T \cdot n_{\text{kv}} \cdot d_h$ números por secuencia.

Proposición 2.2 (Tamaño de la caché KV). *Un modelo de L capas con n_{kv} cabezas de clave y valor de dimensión d_h, que procesa un lote de b secuencias de T tokens con s bytes por número, necesita*

$$M_{\text{KV}} = 2\, L\, n_{\text{kv}}\, d_h\, T\, b\, s \ \textit{bytes}.$$

2.2 La decodificación está limitada por la memoria

El módulo 17 definió la intensidad aritmética de una operación como el cociente entre FLOPs y bytes transferidos, y la comparó con la intensidad crítica del acelerador. El paso de atención de la decodificación tiene una intensidad que no mejora con el lote.

Proposición 2.4 (Intensidad de la atención en la decodificación). *Sea una capa con H cabezas de consulta y n_{kv} cabezas de clave y valor de dimensión d_h, y sea un paso de decodificación de una secuencia cuyo contexto tiene T posiciones. Contando solo los productos $q\,K^\top$ y $a\,V$ y la lectura de la caché, la intensidad aritmética es*

$$I = \frac{2H}{s\, n_{\text{kv}}} \ \textit{FLOP/byte},$$

independiente de T y del tamaño del lote. Con MHA ($n_{\text{kv}} = H$) en bf16, $I = 1$.

La intensidad crítica de la A100 en bf16 es $I^* \approx 156$ FLOP/byte (módulo 17, tema 2). Con MHA, la atención de la decodificación aprovecha menos del 1 % del cómputo disponible, y aumentar el lote no ayuda. Los productos de las matrices de pesos, en cambio, sí ganan intensidad con el lote, porque todas las secuencias comparten los mismos pesos. Shazeer llega a la misma conclusión con otro recuento: en la decodificación incremental con MHA, el cociente entre accesos a memoria y operaciones es $\Theta(n/d + 1/b)$, y el término n/d, debido a la lectura de las claves y los valores, no disminuye con el lote [24, §2.4.1]. Reducir n_{kv} es la forma directa de atacarlo.

2.3 Atención multiconsulta y de consultas agrupadas

Definición 2.5 (MQA y GQA). Sea H divisible por G. La *atención de consultas agrupadas* con G grupos (GQA-G) divide las H cabezas de consulta en G grupos de H/G cabezas consecutivas; todas las cabezas de un grupo comparten una única cabeza de clave y una de valor [1, §2.2]. Con proyecciones $W_Q^{(i)} \in \mathbb{R}^{d \times d_h}$ para $i = 1, \dots, H$ y $W_K^{(g)}, W_V^{(g)} \in \mathbb{R}^{d \times d_h}$ para $g = 1, \dots, G$, la cabeza i usa la clave y el valor del grupo $g(i) = \lceil iG/H \rceil$. El caso $G = 1$ es la *atención multiconsulta* (MQA) [24, §3], y el caso $G = H$ es la atención multicabeza.

Proposición 2.6 (Ahorro de GQA). *Con $n_{\text{kv}} = G$, respecto de MHA:*

(i) la caché KV se divide exactamente por H/G;

(ii) las proyecciones de clave y valor pasan de $2d^2$ a $2dGd_h$ parámetros, y las de consulta y salida no cambian;

(iii) la intensidad de la proposición 2.4 se multiplica por H/G.

Shazeer observó que MQA reduce el tiempo de decodificación por token de 46 a 3,8 microsegundos en su experimento de traducción, con una pérdida de calidad pequeña pero apreciable [24, §4.2 y §4.3]. Ainslie *et al.* proponen GQA como término medio y muestran, además, que un modelo entrenado con MHA puede convertirse en uno con GQA sin entrenarlo desde cero: se promedian las proyecciones de clave y valor de las cabezas de cada grupo y se continúa el entrenamiento durante un 5 % de los pasos originales (*uptraining*) [1, §2.1 y §3.1]. GQA con 8 grupos queda cerca de MHA en calidad y cerca de MQA en velocidad. Llama 2 la usa en sus modelos de 34B y 70B [28, tabla 1], y Llama 3, Mixtral y Grok-1 en todos sus tamaños (cuadro 1.2).

2.4 Atención latente multicabeza

GQA reduce la caché eliminando cabezas de clave y valor. MLA, introducida en DeepSeek-V2, conserva H cabezas, pero guarda en la caché un único vector de baja dimensión por token del que se reconstruyen todas las claves y todos los valores [6, §2.1.2].

Definición 2.8 (MLA sin codificación de posición). Sean $d_c \ll Hd_h$, $W^{DKV} \in \mathbb{R}^{d \times d_c}$ y, para cada cabeza i, $W_i^{UK}, W_i^{UV} \in \mathbb{R}^{d_c \times d_h}$ y $W_i^Q \in \mathbb{R}^{d \times d_h}$. Para el token t con entrada $h_t \in \mathbb{R}^{1 \times d}$,

$$c_t = h_t W^{DKV} \in \mathbb{R}^{1 \times d_c}, \qquad k_{t,i} = c_t W_i^{UK}, \qquad v_{t,i} = c_t W_i^{UV}, \qquad q_{t,i} = h_t W_i^Q.$$

La caché guarda solo los vectores latentes $c_1, \dots, c_T$.

Guardar c_t en lugar de $k_{t,i}$ y $v_{t,i}$ solo ahorra memoria si no hay que reconstruir explícitamente las claves y los valores en cada paso. La factorización lo evita.

Proposición 2.9 (Absorción de las proyecciones). *Con la notación de la definición 2.8, sean $\tilde{q}_{t,i} = q_{t,i}(W_i^{UK})^\top \in \mathbb{R}^{1\times d_c}$ y $a_{t,i,j}$ los pesos de atención de la cabeza i. Entonces*

$$q_{t,i}\, k_{j,i}^\top = \tilde{q}_{t,i}\, c_j^\top, \qquad \sum_{j\le t} a_{t,i,j}\, v_{j,i} = \Big(\sum_{j\le t} a_{t,i,j}\, c_j\Big) W_i^{UV}.$$

En consecuencia, las puntuaciones y la salida de cada cabeza se calculan a partir de los c_j, sin formar $k_{j,i}$ ni $v_{j,i}$; además, W_i^{UK} puede integrarse en la proyección de consulta y W_i^{UV}, en la de salida.

Observación 2.10 (RoPE desacoplada). La absorción falla con RoPE. Con vectores fila, la rotación de la posición m actúa como $x \mapsto xR_m$, y $R_m R_n^\top = R_{m-n}$ (es la versión traspuesta de la identidad del módulo 14, tema 4). Si la consulta y la clave se rotan, la puntuación es $q_{t,i}R_t(c_j W_i^{UK} R_j)^\top = q_{t,i}R_{t-j}(W_i^{UK})^\top c_j^\top$: entre $q_{t,i}$ y c_j aparece una matriz que depende de la distancia $t-j$, y ya no hay una matriz fija que integrar en la consulta. DeepSeek-V2 lo resuelve añadiendo a cada cabeza una componente de dimensión d_h^R que sí se rota y cuya clave k_t^R es compartida por todas las cabezas y se guarda también en la caché [6, §2.1.3]. La caché por token y capa tiene entonces $d_c + d_h^R$ números.

Cuadro 2.1: Números guardados en la caché KV por token, en un modelo de L capas [6, tabla 1].

Mecanismo	Caché por token	DeepSeek-V2 ($L=60$, $H=d_h=128$)
MHA	$2Hd_hL$	1 966 080
GQA-G	$2Gd_hL$	15 360 G
MQA	$2d_hL$	15 360
MLA	$(d_c+d_h^R)L$	$(512+64)\cdot 60 = 34\,560$

Tema 3

Atención local y contexto largo

La atención causal completa evalúa del orden de $T^2/2$ pares de posiciones por capa, y su caché crece linealmente con T. Este tema estudia dos respuestas complementarias. La primera limita a cada token a una ventana de posiciones recientes, de modo que el coste por capa pasa a ser lineal en T y la caché queda acotada; la profundidad del modelo recupera parte del alcance perdido. La segunda permite que un modelo entrenado con un contexto corto funcione con uno más largo, modificando las frecuencias de RoPE. Se sigue a Jiang *et al.* [19, §2] para la ventana deslizante, a Beltagy *et al.* [2, §3.1] para los patrones dispersos, a Chen *et al.* [3, §2] y a Peng *et al.* [22, §2–3] para la extensión del contexto, y las clases 3 y 10 de CS336 [16].

3.1 Atención con ventana deslizante

Definición 3.1 (Atención con ventana deslizante). Sea $W \geq 1$. La *atención con ventana deslizante* de tamaño W usa la máscara

$$M_{ij} = \begin{cases} 0 & \text{si } i - W < j \leq i, \\ -\infty & \text{en otro caso,} \end{cases}$$

de modo que la posición i atiende a las $\min(i, W)$ posiciones $j \in \{\max(1, i - W + 1), \ldots, i\}$, incluida ella misma. Con $W \geq T$ se recupera la máscara causal.

Proposición 3.2 (Pares evaluados). *Para una secuencia de T tokens, la atención causal completa evalúa $T(T+1)/2$ pares (i, j) por capa y cabeza. Con ventana $W \leq T$ evalúa*

$$\frac{W(W+1)}{2} + (T - W)\, W = TW - \frac{W(W-1)}{2}$$

pares. Para $T \gg W$, el cociente entre ambos recuentos es aproximadamente $T/(2W)$.

3.2 Campo receptivo

Una capa con ventana solo mezcla información de W posiciones, pero la capa siguiente recibe vectores que ya resumen otras W, como en una red convolucional.

Teorema 3.3 (Campo receptivo de la atención con ventana). *En una pila de L capas de atención con ventana W, y con conexiones residuales y subcapas posición a posición entre ellas, la salida de la posición i tras la capa L solo puede depender de las entradas en las posiciones $\{i - L(W-1), \ldots, i\}$. El campo receptivo tiene, por tanto, $L(W-1)+1$ posiciones.*

Observación 3.5. El teorema 3.3 da una cota superior de la dependencia, no una garantía de que la información llegue. Un dato situado a 10^5 posiciones de distancia solo puede alcanzar la posición i tras pasar por decenas de capas intermedias, cada una de las cuales lo mezcla con

otros. Por eso la clase 10 de CS336 advierte que la atención local puede perjudicar la calidad y que la solución habitual es intercalar capas locales y globales [16].

3.3 La caché circular

Definición 3.6 (Caché circular). Con ventana W, la *caché circular* de una capa tiene W ranuras $0, \dots, W-1$. Las claves y los valores de la posición $i \geq 0$ se escriben en la ranura $i \bmod W$, sobrescribiendo lo que hubiera [19, §2].

Proposición 3.7 (Corrección de la caché circular). *Tras escribir la posición i, la caché contiene exactamente las claves y los valores de las posiciones* $\max(0, i-W+1), \dots, i$, *que son las que la posición i necesita.*

La memoria de la caché pasa a ser $2Ln_{\text{kv}}d_hWs$ bytes por secuencia, independiente de T. En Mistral 7B ($L=32$, $n_{\text{kv}}=8$, $d_h=128$), en bf16 son 512 MiB, frente a 4 GiB para una secuencia de 32 768 tokens sin ventana: el artículo cita esa reducción de 8× [19, §2].

3.4 Patrones dispersos e híbridos

Longformer combina tres patrones [2, §3.1]: la ventana deslizante; la *ventana dilatada*, en la que cada posición atiende a w posiciones separadas por saltos de tamaño δ, con un campo receptivo del orden de $L\,\delta\,w$; y la *atención global* de unas pocas posiciones elegidas, que atienden a toda la secuencia y a las que atiende toda la secuencia. El coste sigue siendo $O(Tw)$.

Los modelos recientes prefieren alternar capas. La clase 3 de CS336 cita un modelo con una capa de atención completa cada cuatro y el resto con ventana, y la clase 10 recoge un diseño con una capa global cada seis [16].

Proposición 3.8 (Caché de un modelo híbrido). *Sea un modelo de L capas en el que una fracción ρ usa atención completa y el resto ventana W. Con $T \geq W$, su caché es la de un modelo de atención completa multiplicada por* $\rho + (1-\rho)W/T$.

Con $\rho = 1/4$, $W = 4096$ y $T = 131\,072$, el factor es $0{,}25 + 0{,}75/32 = 0{,}2734$: la caché se reduce a algo más de una cuarta parte, dominada por las capas globales.

3.5 RoPE y longitudes de onda

RoPE (módulo 14, tema 4) gira cada par de coordenadas $i \in \{0, \dots, d_h/2-1\}$ de consultas y claves un ángulo $m\theta_i$ en la posición m, con $\theta_i = \beta_0^{-2i/d_h}$ y base $\beta_0 = 10^4$ en el artículo original [27, §3.2.2]. Llama 3 usa $\beta_0 = 5 \cdot 10^5$ [13, tabla 3].

Definición 3.9 (Longitud de onda). La *longitud de onda* del par i es $\lambda_i = 2\pi/\theta_i = 2\pi\beta_0^{2i/d_h}$: el número de posiciones tras el cual la rotación da una vuelta completa [22, ec. 8].

Las longitudes de onda crecen geométricamente desde $\lambda_0 = 2\pi \approx 6{,}283$ hasta $\lambda_{d_h/2-1} = 2\pi\beta_0^{1-2/d_h}$. Con $d_h = 128$ y $\beta_0 = 10^4$, la mayor es 54 410; con $\beta_0 = 5 \cdot 10^5$, unos $2{,}56 \cdot 10^6$. Los pares de longitud de onda corta codifican la posición relativa con precisión; los de longitud

de onda mayor que el contexto de entrenamiento no completan nunca una vuelta durante el entrenamiento, de modo que se comportan casi como una codificación absoluta de la posición.

3.6 Extensión del contexto

Sea un modelo entrenado con contexto L_0 que se quiere usar con $L' > L_0$, y sea $s_e = L'/L_0$. Usar posiciones $m \geq L_0$ sin más (extrapolación) produce ángulos $m\theta_i$ que los pares de onda larga nunca vieron, y la calidad se degrada rápidamente [3, §2.2].

Definición 3.10 (Interpolación de posiciones). La *interpolación de posiciones* (PI) sustituye la posición m por m/s_e antes de aplicar RoPE: el ángulo del par i pasa a ser $m\theta_i/s_e$ [3, §2.3, ec. 4].

Con PI, todo ángulo usado con $m < L'$ es menor que $L_0\theta_i$, es decir, está dentro del intervalo visto en el entrenamiento. Chen *et al.* demuestran que el error al interpolar entre posiciones conocidas está acotado por una cantidad mucho menor que la cota correspondiente a la extrapolación [3, §2.3, teorema 2.1], y extienden modelos LLaMA hasta 32 768 tokens con unos 1000 pasos de ajuste. El inconveniente es que PI comprime también los pares de onda corta, que no lo necesitaban, y la resolución entre posiciones cercanas empeora en un factor s_e.

Definición 3.11 (Cambio de base NTK). La interpolación *consciente del NTK* conserva las posiciones y cambia la base por

$$\beta_0' = \beta_0 \, s_e^{d_h/(d_h-2)}$$

[22, §3.1].

Proposición 3.12 (Efecto del cambio de base). *Con la base β_0' de la definición 3.11, las nuevas frecuencias son $\theta_i' = \theta_i \, s_e^{-2i/(d_h-2)}$. En particular, $\theta_0' = \theta_0$ (el par más rápido no cambia) y $\theta_{d_h/2-1}' = \theta_{d_h/2-1}/s_e$ (el más lento se interpola exactamente como en PI). Las frecuencias intermedias se dividen por factores que crecen geométricamente con i entre 1 y s_e.*

YaRN refina esta idea en dos pasos [22, §3.2 y §3.3]. Primero, decide para cada par según el cociente $r_i = L_0/\lambda_i$, que cuenta cuántas vueltas completa en el contexto original: si $r_i > \beta$ (onda corta) no interpola; si $r_i < \alpha$ (onda larga) interpola como PI; entre ambos valores mezcla linealmente las dos opciones. Para Llama, $\alpha = 1$ y $\beta = 32$. Segundo, multiplica los logits de la atención por un factor de temperatura, implementado escalando consultas y claves por $\sqrt{1/t} = 0{,}1 \ln s_e + 1$ [22, §3.3]. Con $s_e = 16$, $\sqrt{1/t} = 1{,}277$. Los autores extienden Llama 2 a 128 000 tokens con 400 pasos de ajuste [22, §4.1].

Observación 3.14 (Entrenamiento por etapas). La extensión del contexto también puede hacerse durante el preentrenamiento. Llama 3 405B se entrena primero con 8192 tokens y amplía el contexto hasta 128 000 en seis etapas, con unos $8 \cdot 10^{11}$ tokens, al final del preentrenamiento, porque el coste de la atención crece cuadráticamente con la longitud [13, §3.4.2]. El informe de Gemini 1.5 afirma que el modelo recupera información en contextos de hasta 10^7 tokens, pero no publica el mecanismo que lo permite [12, §3.1].

Tema 4

Mezcla de expertos

Un Transformer denso aplica a cada token todos sus parámetros. Una capa de *mezcla de expertos* (MoE) sustituye el MLP por varios MLP alternativos y por un enrutador que elige unos pocos para cada token, de modo que los parámetros crecen con el número de expertos y el cómputo por token solo con el número de expertos elegidos. Este tema formula la capa, distingue parámetros totales y activos, estudia el problema del balanceo de carga y sus soluciones, y reconstruye los recuentos publicados de varios modelos. Se sigue la clase 4 de CS336 [16], los artículos de Shazeer *et al.* [26, §2 y §4], Fedus *et al.* [10, §2] y Jiang *et al.* [18, §2], y los informes de DeepSeek-V2 y DeepSeek-V3 [6, §2.2], [7, §2.1.2].

4.1 La capa de mezcla de expertos

Definición 4.1 (Capa MoE con enrutado top-k). Sean $E_1, \dots, E_N : \mathbb{R}^{1\times d} \to \mathbb{R}^{1\times d}$ MLP con la misma forma (los *expertos*), $W_r \in \mathbb{R}^{d\times N}$ la matriz del *enrutador* y $1 \le k \le N$. Para un token x, sean $z = xW_r \in \mathbb{R}^N$ sus logits y $\mathcal{T}(x)$ el conjunto de los índices de los k logits mayores. La salida de la capa es

$$y = \sum_{i\in\mathcal{T}(x)} g_i(x)\, E_i(x), \qquad g_i(x) = \frac{e^{z_i}}{\sum_{j\in\mathcal{T}(x)} e^{z_j}}.$$

Los expertos que no están en $\mathcal{T}(x)$ no se evalúan. Esta es la formulación de Mixtral, $G(x) = \mathrm{softmax}(\mathrm{TopK}(xW_r))$ [18, §2.1].

Hay variantes en el orden de las operaciones. Shazeer *et al.* también normalizan solo entre los elegidos, pero añaden ruido gaussiano a los logits, con una desviación aprendida, $H(x)_i = (xW_r)_i + \xi_i \,\mathrm{softplus}\big((xW_{\text{ruido}})_i\big)$ con $\xi_i \sim \mathcal{N}(0,1)$, para favorecer la exploración y el balanceo [26, §2.1]. El Switch Transformer usa $k = 1$ y pondera la salida por la probabilidad de la softmax sobre los N expertos, sin renormalizar [10, §2.1]. La clase 4 de CS336 recoge que DeepSeek-V1 y V2, Grok y Qwen aplican la softmax antes del top-k, y que Mixtral, DBRX y DeepSeek-V3 normalizan después [16].

Proposición 4.2 (Renormalización). *Sean $p = \mathrm{softmax}(z)$ las probabilidades sobre los N expertos y g los pesos de la definición 4.1. Para $i \in \mathcal{T}(x)$, $g_i = p_i / \sum_{j\in\mathcal{T}(x)} p_j \ge p_i$, con igualdad para todo i si y solo si $k = N$. Además, el orden de los expertos elegidos es el mismo con g que con p.*

4.2 Parámetros totales y activos

Proposición 4.4 (Parámetros totales y activos). *Sea un modelo con L capas MoE de N expertos de P_E parámetros cada uno, k de ellos activos por token, y sea P_0 el número*

de parámetros compartidos por todos los tokens (atención, embeddings*, normalizaciones y enrutadores). Entonces*

$$N_{\text{tot}} = P_0 + LNP_E, \qquad N_{\text{act}} = P_0 + LkP_E.$$

El cómputo por token en la pasada hacia delante es de unos $2N_{\text{act}}$ *FLOPs y el de entrenamiento con* D *tokens,* $C \approx 6N_{\text{act}}D$*, mientras que la memoria de los pesos es proporcional a* N_{tot}*.*

4.3 Balanceo de carga

Si un experto empieza siendo algo mejor, el enrutador le envía más tokens, recibe más gradiente, mejora más y acaba acaparando el tráfico; los demás apenas se entrenan. Shazeer *et al.* ya observaron este colapso y lo combatieron con una pérdida sobre el coeficiente de variación de la importancia de cada experto, $\text{Importance}(X) = \sum_{x \in X} G(x)$ [26, §4]. La formulación más usada es la del Switch Transformer.

Definición 4.6 (Pérdida de balanceo). Sea un lote $\mathcal{B}$ de tokens. Para cada experto i, sean f_i la fracción de las asignaciones del lote que recibe y $P_i = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} p_i(x)$ la probabilidad media que le da el enrutador. La *pérdida de balanceo* es

$$\mathcal{L}_{\text{bal}} = N \sum_{i=1}^{N} f_i P_i,$$

y se suma a la entropía cruzada con un coeficiente α; el Switch Transformer usa $\alpha = 10^{-2}$ [10, §2.2, ecs. 4–6].

Las fracciones f_i proceden de una selección discreta y no son derivables; el gradiente fluye solo a través de P_i. La clase 4 de CS336 lo resume así: la derivada respecto de $p_i(x)$ es proporcional al número de tokens asignados al experto i, de modo que un experto más usado ve reducida su probabilidad con más fuerza [16].

Teorema 4.7 (Cotas de la pérdida de balanceo). *Si* $f = P$ *(el tráfico real coincide con la probabilidad media), entonces* $1 \le \mathcal{L}_{\text{bal}} \le N$*. La cota inferior se alcanza si y solo si* $P_i = 1/N$ *para todo* i*, y la superior, si y solo si un experto recibe todo el tráfico.*

Cuadro 4.1: Valores de $\mathcal{L}_{\text{bal}}$ con $N = 4$.

Situación	f	P	$\mathcal{L}_{\text{bal}}$
Reparto uniforme	$(\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})$	$(\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})$	1
Desequilibrio	(0,5; 0,3; 0,1; 0,1)	(0,4; 0,3; 0,2; 0,1)	1,28
Colapso	$(1, 0, 0, 0)$	$(1, 0, 0, 0)$	4

Observación 4.8 (Balanceo sin pérdida auxiliar). DeepSeek-V3 sustituye casi por completo la pérdida auxiliar por un sesgo b_i por experto que solo interviene en la selección: los expertos se eligen según $s_i(x) + b_i$, pero se ponderan con $s_i(x)$, y al final de cada paso b_i disminuye en una cantidad fija γ si el experto ha estado sobrecargado y aumenta en γ si ha estado infrautilizado [7, §2.1.2]. Mantiene, con un coeficiente muy pequeño, una pérdida de balanceo por secuencia. DeepSeek-V3 usa además afinidades sigmoides, $s_i(x) = \sigma(u\, e_i^{\top})$, en lugar de una softmax.

4.4 Capacidad y descarte de tokens

En un acelerador, cada experto procesa un tensor de tamaño fijo, de modo que el número de tokens que puede recibir por lote está acotado.

Definición 4.9 (Capacidad de un experto). Con un lote de B_{t} tokens, N expertos, k asignaciones por token y un *factor de capacidad* $c \geq 1$, la *capacidad* de cada experto es

$$C_{\mathrm{exp}} = c \cdot \frac{k\, B_{\mathrm{t}}}{N}$$

tokens [10, §2.2, ec. 3]. Las asignaciones que exceden la capacidad se *descartan*: para ellas, ese experto no se evalúa.

Un token descartado en todos sus expertos sale de la capa con su vector intacto, porque el bloque calcula $h + \mathrm{FFN}(\cdot)$ y la contribución del MLP es nula: no se pierde la información del token, sino la transformación de esa capa. Con $B_{\mathrm{t}} = 1024$, $N = 8$, $k = 1$ y $c = 1{,}25$, la capacidad es $1{,}25 \cdot 128 = 160$ tokens por experto. Un factor mayor descarta menos tokens, pero reserva más memoria y cómputo para posiciones de relleno. La clase 4 de CS336 señala una consecuencia práctica: como el descarte se decide por lote, las peticiones de otros usuarios pueden provocar el descarte de los tokens propios, lo que añade no determinismo a la inferencia [16].

4.5 Expertos de grano fino y compartidos

DeepSeekMoE divide cada experto en m expertos más pequeños y activa m veces más, de modo que los parámetros y el cómputo no cambian, y reserva algunos expertos *compartidos* que procesan todos los tokens [6, §2.2.1]. DeepSeek-V2 usa 2 expertos compartidos y 160 enrutados, de los que activa 6; DeepSeek-V3, 1 compartido y 256 enrutados, de los que activa 8 [6, 7].

Proposición 4.10 (Combinaciones de expertos). *Si cada uno de N expertos se divide en m expertos de tamaño $1/m$ y se activan mk de ellos, el número de combinaciones posibles pasa de $\binom{N}{k}$ a $\binom{mN}{mk}$, con los mismos parámetros totales y activos.*

Con $N = 8$, $k = 2$ y $m = 4$, se pasa de $\binom{8}{2} = 28$ combinaciones a $\binom{32}{8} = 10\,518\,300$. La clase 4 de CS336 recoge que las ablaciones de DeepSeek muestran ganancias de los expertos de grano fino y de los compartidos, mientras que las de OLMoE confirman las primeras pero no las segundas [16].

4.6 Casos públicos

Observación 4.11 (Lo que no es público). El informe de Gemini 1.5 describe el modelo como un Transformer MoE disperso, pero no da el número de expertos ni el de parámetros [12, §3.1]. Anthropic no ha publicado la arquitectura de Claude, y xAI no ha publicado la de las versiones de Grok posteriores a Grok-1. Nada de lo que se dice en este tema debe atribuirse a esos modelos.

Observación 4.12 (Paralelismo de expertos). Las MoE añaden una forma de paralelismo a las del módulo 17: cada dispositivo guarda un subconjunto de expertos y los tokens viajan al dispositivo de su experto y vuelven mediante una comunicación de todos con todos (*all-to-all*).

Cuadro 4.2: Modelos MoE con información publicada por sus autores.

Modelo	Expertos y enrutado	Parámetros
Shazeer *et al.* (2017) [26]	hasta 131 072 expertos entre capas LSTM, top-k con ruido	hasta $1{,}37 \cdot 10^{11}$ en la capa MoE
Switch-C (2021) [10]	2048 expertos, top-1	$1{,}571 \cdot 10^{12}$ (tabla 9)
Mixtral 8x7B (2024) [18]	8 expertos, top-2	$4{,}7 \cdot 10^{10}$ totales, $1{,}3 \cdot 10^{10}$ activos
Grok-1 (2024) [30]	8 expertos, 2 por token	$3{,}14 \cdot 10^{11}$
DeepSeek-V2 (2024) [6]	2 compartidos y 160 enrutados, 6 activos	$2{,}36 \cdot 10^{11}$ totales, $2{,}1 \cdot 10^{10}$ activos
DeepSeek-V3 (2024) [7]	1 compartido y 256 enrutados, 8 activos	$6{,}71 \cdot 10^{11}$ totales, $3{,}7 \cdot 10^{10}$ activos
Gemini 1.5 Pro (2024) [12]	«Transformer de mezcla de expertos dispersa»	no publicado

DeepSeek-V2 limita el número de dispositivos a los que puede ir cada token para acotar esa comunicación [6, §2.2.2].

Tema 5

Modelos de espacio de estados

La atención guarda todo el pasado y lo consulta en cada paso; una red recurrente lo resume en un estado de tamaño fijo. Los modelos de espacio de estados (SSM) son redes recurrentes lineales con una estructura heredada de la teoría de sistemas: se definen en tiempo continuo, se discretizan y pueden calcularse como recurrencia, con coste constante por token en la generación, o como convolución, en paralelo durante el entrenamiento. Mamba los hace selectivos, haciendo que sus parámetros dependan de la entrada, y recupera el cálculo paralelo mediante un barrido asociativo. Se sigue a Gu, Goel y Ré [15, §2], a Gu y Dao [14, §2 y §3], la parte de la clase 10 de CS336 dedicada a las alternativas al Transformer [16] y la sesión de A. Gu en el seminario CS25 [11]. Los sistemas discretos y su respuesta al impulso siguen el curso 6.003 del MIT [21].

5.1 Sistemas lineales continuos y su discretización

Definición 5.1 (Modelo de espacio de estados). Sean $A \in \mathbb{R}^{n\times n}$, $B \in \mathbb{R}^{n\times 1}$ y $C \in \mathbb{R}^{1\times n}$. El *modelo de espacio de estados* continuo con entrada escalar $x(t)$, estado $h(t) \in \mathbb{R}^n$ y salida $y(t)$ es

$$h'(t) = Ah(t) + Bx(t), \qquad y(t) = Ch(t)$$

[14, ecs. 1a y 1b]. Un modelo de lenguaje aplica un SSM independiente a cada una de las d componentes de la secuencia de vectores.

Para procesar una secuencia se fija un paso $\Delta > 0$ y se supone que la entrada es constante en cada intervalo de longitud Δ (*retención de orden cero*).

Proposición 5.2 (Discretización por retención de orden cero). *Sea $n = 1$, con $A = a \neq 0$ y $B = \beta$. Si $x(\tau) = x_t$ es constante en $[t\Delta, (t+1)\Delta)$, entonces $h_{t+1} = \bar{A}h_t + \bar{B}x_t$, con*

$$\bar{A} = e^{a\Delta}, \qquad \bar{B} = \frac{e^{a\Delta} - 1}{a}\,\beta.$$

Si $a < 0$, entonces $0 < \bar{A} < 1$. En el caso matricial, $\bar{A} = \exp(\Delta A)$ y $\bar{B} = (\Delta A)^{-1}\big(\exp(\Delta A) - I\big)\Delta B$ [14, ec. 4].

S4 usa en su lugar la discretización bilineal, $\bar{A} = (I - \frac{\Delta}{2}A)^{-1}(I + \frac{\Delta}{2}A)$ y $\bar{B} = (I - \frac{\Delta}{2}A)^{-1}\Delta B$ [15, §2.3].

5.2 Recurrencia y convolución

Definición 5.4 (SSM discreto). El SSM discreto con matrices $(\bar{A}, \bar{B}, C)$ y estado inicial $h_0 = 0$ transforma la sucesión $(x_t)_{t\geq 1}$ en

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \qquad y_t = Ch_t$$

[14, ecs. 2a y 2b].

Proposición 5.5 (Forma convolucional). *Si $(\bar{A}, \bar{B}, C)$ no dependen de t, la salida del SSM discreto es la convolución causal*

$$y_t = \sum_{j=0}^{t-1} \bar{K}_j \, x_{t-j}, \qquad \bar{K}_j = C \bar{A}^j \bar{B},$$

y $\bar{K}$ es la respuesta del sistema al impulso unitario ($x_1 = 1$ y $x_t = 0$ para $t > 1$).

Las dos formas tienen costes complementarios. La recurrencia cuesta $O(n)$ operaciones por token y un estado de tamaño fijo, como una red recurrente, pero es secuencial. La convolución se calcula en paralelo sobre toda la secuencia, por ejemplo con la transformada rápida de Fourier en $O(T \log T)$, que es como S4 entrena [15, §2.4]. Gu y Dao llaman a esta propiedad *invariancia temporal lineal* [14, §2].

5.3 Selección: el modelo de Mamba

La invariancia temporal impide que el modelo decida, según el contenido, qué recordar y qué olvidar: el núcleo $\bar{K}$ trata igual cada token. Gu y Dao argumentan que esta es la razón por la que los SSM anteriores eran débiles en lenguaje, y proponen que los parámetros dependan de la entrada [14, §3.1 y §3.2].

Definición 5.7 (SSM selectivo). Un *SSM selectivo* usa en el instante t las matrices $B_t = s_B(x_t)$, $C_t = s_C(x_t)$ y el paso $\Delta_t = \mathrm{softplus}\big(p + s_\Delta(x_t)\big)$, donde s_B, s_C y s_Δ son proyecciones lineales de la entrada y p es un parámetro, y discretiza con la proposición 5.2 en cada instante [14, §3.2, algoritmo 2]. La matriz A es fija y diagonal.

Teorema 5.8 (Selección y puertas). *Sea un SSM selectivo escalar con $A = -1$, $B = 1$ y $\Delta_t = \mathrm{softplus}(z_t)$, donde z_t es una función lineal de x_t. Su recurrencia es*

$$h_t = (1 - g_t)\, h_{t-1} + g_t \, x_t, \qquad g_t = \sigma(z_t),$$

la de una red recurrente con una puerta que interpola entre conservar el estado y sustituirlo por la entrada [14, §3.5.1, teorema 1].

5.4 El barrido paralelo

Con parámetros dependientes de t, el coeficiente de x_{t-j} en h_t es $\bar{A}_t \cdots \bar{A}_{t-j+1} \bar{B}_{t-j}$, que depende de t: ya no hay núcleo fijo ni convolución. La linealidad en el estado basta, sin embargo, para paralelizar.

Proposición 5.10 (Asociatividad de la composición de pasos). *Sea $\phi_t(h) = a_t h + b_t$ el paso t de una recurrencia escalar, representado por el par (a_t, b_t). Se define*

$$(a, b) \bullet (a', b') = (a' a, \;\; a' b + b').$$

Entonces $(\phi_{t'} \circ \phi_t)$ está representado por $(a_t, b_t) \bullet (a_{t'}, b_{t'})$, la operación $\bullet$ es asociativa y, con $h_0 = 0$, h_t es la segunda componente de $(a_1, b_1) \bullet \cdots \bullet (a_t, b_t)$.

Por la proposición 5.10, los estados $h_1, \ldots, h_T$ son los prefijos de un producto asociativo, que un algoritmo de barrido paralelo calcula con profundidad $O(\log T)$ y trabajo $O(T)$. Mamba combina este barrido con la fusión de las operaciones en la memoria rápida de la GPU y el recálculo de los estados intermedios en la pasada hacia atrás, sin escribir nunca el estado expandido en la memoria principal [14, §3.3.2]. Con estado vectorial y A diagonal, cada componente es una recurrencia escalar independiente.

5.5 Comparación con la atención

Cuadro 5.1: Atención causal frente a un SSM selectivo, por capa, con T tokens y dimensión d.

	Atención	SSM selectivo
Coste del entrenamiento	$O(T^2 d)$	$O(Td)$ por cada dimensión de estado
Memoria en la generación	caché que crece con T	estado de tamaño fijo
Coste por token generado	$O(Td)$	independiente de T
Recuperar un dato exacto lejano	directo	limitado por la compresión del estado

Gu y Dao informan de un rendimiento de inferencia unas cinco veces mayor que el de un Transformer de tamaño similar [14, §4.5]. La clase 10 de CS336 señala la debilidad complementaria: los SSM anteriores a Mamba resolvían mal las tareas de recuperación asociativa, importantes en lenguaje, y los diseños recientes son híbridos que conservan algunas capas de atención; Jamba, por ejemplo, intercala capas Transformer y Mamba en proporción $1:7$ dentro de una MoE de $5{,}2 \cdot 10^{10}$ parámetros [16, 20]. No hay información pública de que Claude, Gemini o Grok usen SSM.

Tema 6

Multimodalidad: imágenes y audio como tokens

Un Transformer opera sobre secuencias de vectores de dimensión d y no depende de cómo se hayan obtenido. Para que procese imágenes o audio basta con una aplicación que convierta la señal en una secuencia de vectores, el análogo de la tabla de *embeddings* para el texto. Este tema estudia las dos construcciones más citadas: el Vision Transformer, que divide la imagen en parches y proyecta cada uno linealmente, y los codificadores de audio, que parten de un espectrograma o de un códec neuronal que produce tokens discretos. Se sigue a Dosovitskiy *et al.* [9, §3.1], la §11.8 de Zhang *et al.* [32], a Radford *et al.* [23, §2.1 y §2.2] y a Zeghidour *et al.* [31], junto con el informe técnico de Gemini 1.5 [12, §3.1].

6.1 El Vision Transformer

Definición 6.1 (Incrustación por parches). Sea una imagen $X \in \mathbb{R}^{H_{\text{img}} \times W_{\text{img}} \times C_{\text{in}}}$ y sea P un divisor de H_{img} y de W_{img}. Se divide la imagen en $N_p = H_{\text{img}} W_{\text{img}} / P^2$ parches de $P \times P$ píxeles, se aplana cada uno en un vector $x_p^{(k)} \in \mathbb{R}^{1 \times P^2 C_{\text{in}}}$ y se define

$$z_0 = \left[x_{\text{clase}};\ x_p^{(1)} E;\ x_p^{(2)} E;\ \ldots;\ x_p^{(N_p)} E \right] + E_{\text{pos}},$$

con $E \in \mathbb{R}^{P^2 C_{\text{in}} \times d}$, un vector aprendido $x_{\text{clase}} \in \mathbb{R}^{1 \times d}$ y posiciones aprendidas $E_{\text{pos}} \in \mathbb{R}^{(N_p+1) \times d}$ [9, §3.1, ec. 1]. La secuencia z_0, de $N_p + 1$ filas, entra en un Transformer codificador.

Proposición 6.2 (Tokens y coste de un ViT). *Con la notación de la definición 6.1:*

(i) la proyección E tiene $P^2 C_{\text{in}} d$ parámetros, independientemente del tamaño de la imagen;

(ii) si el lado de la imagen se multiplica por κ con P fijo, el número de parches se multiplica por κ^2, el coste de las capas lineales de cada bloque por unos κ^2 y el de los productos de la atención por unos κ^4.

Cuadro 6.1: Número de parches y dimensión de cada parche aplanado para imágenes RGB ($C_{\text{in}} = 3$).

Imagen	Parche	N_p	$P^2 C_{\text{in}}$
224×224	16×16	$14^2 = 196$	768
384×384	16×16	$24^2 = 576$	768
224×224	32×32	$7^2 = 49$	3072

El modelo ViT-Base tiene 12 capas, $d = 768$, un MLP de dimensión 3072 y 12 cabezas,

con unos $8{,}6 \cdot 10^7$ parámetros [9, tabla 1]. Con parches de 16 y una imagen de 224×224, la secuencia tiene $196 + 1 = 197$ tokens y E tiene $768 \cdot 768 = 589\,824$ parámetros.

Observación 6.3 (Parches y convoluciones). Aplicar E a cada parche es lo mismo que aplicar una convolución con d filtros de tamaño $P \times P$ y paso P: cada filtro es una columna de E reordenada como un tensor $P \times P \times C_{\text{in}}$, y el paso igual al tamaño hace que los parches no se solapen. Es la forma en que suele implementarse.

6.2 Audio

El audio es una señal unidimensional muestreada a miles de muestras por segundo: a 16 kHz, treinta segundos son 480 000 muestras, demasiadas para usarlas como tokens. Hay dos formas habituales de reducirlas.

La segunda vía convierte el audio en enteros. Un *códec neuronal* como SoundStream [31] o EnCodec [8] consta de un codificador convolucional que produce un vector por cada fotograma de unos milisegundos, un cuantizador que sustituye ese vector por índices de diccionarios finitos y un decodificador que reconstruye la onda. SoundStream usa un cuantizador vectorial residual: el primer diccionario aproxima el vector, el segundo cuantiza el error del primero, y así sucesivamente.

Proposición 6.5 (Tasa de un cuantizador vectorial residual). *Un códec que produce ν fotogramas por segundo y cuantiza cada uno con N_q diccionarios de K entradas genera νN_q tokens por segundo y una tasa de $\nu N_q \log_2 K$ bits por segundo.*

Con valores ilustrativos, $\nu = 50$ Hz, $N_q = 8$ y $K = 1024$, se obtienen 400 tokens por segundo y 4 kbit/s, dentro del intervalo de 3 a 18 kbit/s que cubre SoundStream [31]. Con estos tokens, el audio se modela con el mismo objetivo de predicción del siguiente token del módulo 1.

6.3 Fusión con el modelo de lenguaje

Hay dos formas de unir los vectores de otra modalidad con los de texto. En la *fusión con adaptador*, un codificador separado (un ViT, por ejemplo) produce vectores que una proyección lineal o un pequeño MLP lleva a la dimensión d del modelo de lenguaje, y se insertan en la secuencia de texto. En la *fusión nativa*, el modelo se entrena desde el principio con secuencias en las que se intercalan tokens de varias modalidades.

Observación 6.6 (Lo que es público). El informe de Gemini 1.5 afirma que el modelo es multimodal de forma nativa y admite datos de distintas modalidades intercalados en una misma secuencia: texto, imágenes, audio y vídeo, con ejemplos de 107 horas de audio y 10,5 horas de vídeo a un fotograma por segundo [12, §3.1]. No describe sus codificadores. Claude y las versiones de Grok posteriores a Grok-1 aceptan imágenes según la documentación pública de sus productos, pero sus codificadores tampoco son públicos; Grok-1, el único con pesos publicados, es un modelo de texto [30]. Lo que sí puede afirmarse en general es que, sea cual sea el codificador, las demás modalidades acaban como vectores en el mismo flujo residual que el texto.

Bibliografía

[1] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón y S. Sanghai. GQA: Training generalized multi-query Transformer models from multi-head checkpoints. En *Proceedings of EMNLP*, 2023. https://arxiv.org/abs/2305.13245

[2] I. Beltagy, M. E. Peters y A. Cohan. Longformer: The long-document Transformer. Prepublicación, 2020. https://arxiv.org/abs/2004.05150

[3] S. Chen, S. Wong, L. Chen e Y. Tian. Extending context window of large language models via position interpolation. Prepublicación, 2023. https://arxiv.org/abs/2306.15595

[4] Carnegie Mellon University. 11-711: Advanced Natural Language Processing. Otoño de 2024. Temario: mecanismos de atención, arquitecturas Transformer, preentrenamiento y multimodalidad. https://www.phontron.com/class/anlp-fall2024/

[5] Carnegie Mellon University. 11-868: Large Language Model Systems. Primavera de 2025. Temario: entrenamiento distribuido, dispersión (MoE), compresión y sistemas de inferencia. https://llmsystem.github.io/llmsystem2025spring/

[6] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. Informe técnico, 2024. https://arxiv.org/abs/2405.04434

[7] DeepSeek-AI. DeepSeek-V3 technical report. Informe técnico, 2024. https://arxiv.org/abs/2412.19437

[8] A. Défossez, J. Copet, G. Synnaeve e Y. Adi. High fidelity neural audio compression. Prepublicación, 2022. https://arxiv.org/abs/2210.13438

[9] A. Dosovitskiy *et al.* An image is worth 16×16 words: Transformers for image recognition at scale. En *International Conference on Learning Representations (ICLR)*, 2021. https://arxiv.org/abs/2010.11929

[10] W. Fedus, B. Zoph y N. Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23:1–40, 2022. https://arxiv.org/abs/2101.03961

[11] S. Feng, K. Singh, M. C. Frank y C. D. Manning. CS25: Transformers United, edición V6. Stanford University, primavera. Sesiones de A. Gu («On the Tradeoffs of State Space Models and Transformers»), N. Tazi («The Ultra-Scale Talk: Scaling Training to Thousands of GPUs») y V. Lin («From Language Models to Native Multimodal Intelligence»). https://web.stanford.edu/class/cs25/

[12] Gemini Team, Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. Informe técnico, 2024. https://arxiv.org/abs/2403.05530

[13] A. Grattafiori *et al.* (Llama Team, AI @ Meta). The Llama 3 herd of models. Informe técnico, 2024. https://arxiv.org/abs/2407.21783

[14] A. Gu y T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. Prepublicación, 2023 (revisada en 2024). https://arxiv.org/abs/2312.00752

[15] A. Gu, K. Goel y C. Ré. Efficiently modeling long sequences with structured state spaces. En *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2111.00396

[16] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2025. Clases 3 («Architectures, hyperparameters»), 4 («Mixture of experts») y 10 («Inference»), transparencias y código publicados. https://github.com/stanford-cs336/spring2025-lectures

[17] Equipo docente de CS336. CS336: Language Modeling from Scratch. Tarea 1, «Basics», versión 26.0.3. Stanford University, primavera de 2026. §3.4.1 (RMSNorm), §3.4.2 (MLP SwiGLU) y §7.3 (ablaciones de arquitectura). https://github.com/stanford-cs336/assignment1-basics

[18] A. Q. Jiang *et al.* Mixtral of experts. Prepublicación, 2024. https://arxiv.org/abs/2401.04088

[19] A. Q. Jiang *et al.* Mistral 7B. Prepublicación, 2023. https://arxiv.org/abs/2310.06825

[20] O. Lieber *et al.* Jamba: A hybrid Transformer-Mamba language model. Prepublicación, 2024. Citado a través de la clase 10 de CS336 [16]. https://arxiv.org/abs/2403.19887

[21] Massachusetts Institute of Technology. 6.003: Signals and Systems. MIT OpenCourseWare, otoño de 2011. *Homework* 2, problemas 1 («Finding outputs») y 6 («Unit-sample response»). https://ocw.mit.edu/courses/6-003-signals-and-systems-fall-2011/

[22] B. Peng, J. Quesnelle, H. Fan y E. Shippole. YaRN: Efficient context window extension of large language models. En *International Conference on Learning Representations (ICLR)*, 2024. https://arxiv.org/abs/2309.00071

[23] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey e I. Sutskever. Robust speech recognition via large-scale weak supervision. En *International Conference on Machine Learning (ICML)*, 2023. https://arxiv.org/abs/2212.04356

[24] N. Shazeer. Fast Transformer decoding: One write-head is all you need. Prepublicación, 2019. https://arxiv.org/abs/1911.02150

[25] N. Shazeer. GLU variants improve Transformer. Prepublicación, 2020. https://arxiv.org/abs/2002.05202

[26] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton y J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. En *International Conference on Learning Representations (ICLR)*, 2017. https://arxiv.org/abs/1701.06538

[27] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen e Y. Liu. RoFormer: Enhanced Transformer with rotary position embedding. Prepublicación, 2021. Publicado en *Neurocomputing*, 568, 2024. https://arxiv.org/abs/2104.09864

[28] H. Touvron *et al.* Llama 2: Open foundation and fine-tuned chat models. Prepublicación, 2023. https://arxiv.org/abs/2307.09288

[29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser e I. Polosukhin. Attention is all you need. En *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017. https://arxiv.org/abs/1706.03762

[30] xAI. Grok-1: código y pesos publicados (archivos `README.md`, `run.py` y `model.py`). 2024. Licencia Apache 2.0. https://github.com/xai-org/grok-1

[31] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund y M. Tagliasacchi. SoundStream: An end-to-end neural audio codec. Prepublicación, 2021. https://arxiv.org/abs/2107.03312

[32] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. Cap. 11, «Attention Mechanisms and Transformers» (§11.6 «Self-Attention and Positional Encoding» y §11.8 «Transformers for Vision»). https://d2l.ai/

[33] B. Zhang y R. Sennrich. Root mean square layer normalization. En *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. https://arxiv.org/abs/1910.07467